Chapter 5: XGBoost Unveiled

In this chapter, you will finally see **Extreme Gradient Boosting**, or **XGBoost**, as it is. XGBoost is presented in the context of the machine learning narrative that we have built up, from decision trees to gradient boosting. The first half of the chapter focuses on the theory behind the distinct advancements that XGBoost brings to tree ensemble algorithms. The second half focuses on building XGBoost models within the **Higgs Boson Kaggle Competition**, which unveiled XGBoost to the world.

Specifically, you will identify speed enhancements that make XGBoost faster, discover how XGBoost handles missing values, and learn the mathematical derivation behind XGBoost's **regularized parameter selection**. You will establish model templates for building XGBoost classifiers and regressors. Finally, you will look at the **Large Hadron Collider**, where the Higgs boson was discovered, where you will weigh data and make predictions using the original XGBoost Python API.

This chapter covers the following main topics:

- Designing XGBoost
- Analyzing XGBoost parameters
- Building XGBoost models
- Finding the Higgs boson – case study

Designing XGBoost

XGBoost is a significant upgrade from gradient boosting. In this section, you will identify the key features of XGBoost that distinguish it from gradient boosting and other tree ensemble algorithms.

Historical narrative

With the acceleration of big data, the quest to find awesome machine learning algorithms to produce accurate, optimal predictions began. Decision trees produced machine learning models that were too accurate and failed to generalize well to new data. Ensemble methods proved more effective by combining many decision trees via **bagging** and **boosting**. A leading algorithm that emerged from the tree ensemble trajectory was gradient boosting.

The consistency, power, and outstanding results of gradient boosting convinced Tianqi Chen from the University of Washington to enhance its capabilities. He called the new algorithm XGBoost, short for **Extreme Gradient Boosting**. Chen's new form of gradient boosting included built-in regularization and impressive gains in speed.

After finding initial success in Kaggle competitions, in 2016, Tianqi Chen and Carlos Guestrin authored *XGBoost: A Scalable Tree Boosting System* to present their algorithm to the larger machine learning community. You can check out the original paper at **https://arxiv.org/pdf/1603.02754.pdf**. The key points are summarized in the following section.

Design features

As indicated in ***Chapter 4***, *From Gradient Boosting to XGBoost*, the need for faster algorithms is evident when dealing with big data. The *Extreme* in *Extreme Gradient Boosting* means pushing computational limits to the extreme. Pushing computational limits requires knowledge not just of model-building but also of disk-reading, compression, cache, and cores.

Although the focus of this book remains on building XGBoost models, we will take a glance under the hood of the XGBoost algorithm to distinguish key advancements, such as handling missing values, speed gains, and accuracy gains that make XGBoost faster, more accurate, and more desirable. Let's look at these key advancements next.

Handling missing values

You spent significant time in ***Chapter 1***, *Machine Learning Landscape*, practicing different ways to correct **null values**. This is an essential skill for all machine learning practitioners.

XGBoost, however, is capable of handling missing values for you. There is a `missing` hyperparameter that can be set to any value. When given a missing data point, XGBoost scores different split options and chooses the one with the best results.

Gaining speed

XGBoost was specifically designed for speed. Speed gains allow machine learning models to build more quickly which is especially important when dealing with millions, billions, or trillions of rows of data. This is not uncommon in the world of big data, where each day, industry and science accumulate more data than ever before. The following new design features give XGBoost a big edge in speed over comparable ensemble algorithms:

- **Approximate split-finding algorithm**
- **Sparsity aware split-finding**
- **Parallel computing**
- **Cache-aware access**
- **Block compression and sharding**

Let's learn about these features in a bit more detail.

Approximate split-finding algorithm

Decision trees need optimal splits to produce optimal results. A *greedy algorithm* selects the best split at each step and does not backtrack to look at previous branches. Note that decision tree splitting is usually performed in a greedy manner.

XGBoost presents an exact greedy algorithm in addition to a new approximate split-finding algorithm. The split-finding algorithm uses **quantiles**, percentages that split data, to propose candidate splits. In a global proposal, the same quantiles are used throughout the entire training, and in a local proposal, new quantiles are provided for each round of splitting.

A previously known algorithm, **quantile sketch**, works well with equally weighted datasets. XGBoost presents a novel weighted quantile sketch based on merging and pruning with a theoretical guarantee. Although the mathematical details of this algorithm are beyond the scope of this book, you are encouraged to check out the appendix of the original XGBoost paper at **https://arxiv.org/pdf/1603.02754.pdf**.

Sparsity-aware split finding

Sparse data occurs when the majority of entries are 0 or null. This may occur when datasets consist primarily of null values or when they have been **one-hot encoded**. In ***Chapter 1***, *Machine Learning Landscape*, you used `pd.get_dummies` to transform **categorical columns** into **numerical columns**. This resulted in a larger dataset with many values of 0. This method of converting categorical columns into numerical columns, where 1 indicates presence and 0 indicates absence, is generally referred to as one-hot encoding. You will gain practice with one-hot-encoding in ***Chapter 10***, *XGBoost Model Deployment*.

Sparse matrices are designed to only store data points with non-zero and non-null values. This saves valuable space. A sparsity-aware split indicates that when looking for splits, XGBoost is faster because its matrices are sparse.

According to the original paper, *XGBoost: A Scalable Tree Boosting System*, the sparsity-aware split-finding algorithm performed 50 times faster than the standard approach on the **All-State-10K** dataset.

Parallel computing

Boosting is not ideal for **parallel computing** since each tree depends on the results of the previous tree. There are opportunities, however, where parallelization may take place.

Parallel computing occurs when multiple computational units are working together on the same problem at the same time. XGBoost sorts and compresses the data into blocks. These blocks may be distributed to multiple machines, or to external memory (out of core).

Sorting the data is faster with blocks. The split-finding algorithm takes advantage of blocks and the search for quantiles is faster due to blocks. In each of these cases, XGBoost provides parallel computing to expedite the model-building process.

Cache-aware access

The data on your computer is separated into **cache** and **main memory**. The cache, what you use most often, is reserved for high-speed memory. The data that you use less often is held back for lower-speed memory. Different cache levels have different orders of magnitude of latency, as outlined here: **https://gist.github.com/jboner/2841832**.

When it comes to gradient statistics, XGBoost uses **cache-aware prefetching**. XGBoost allocates an internal buffer, fetches the gradient statistics, and performs accumulation with mini batches. According to *XGBoost: A Scalable Tree Boosting System*, prefetching lengthens read/write dependency and reduces runtimes by approximately 50% for datasets with a large number of rows.

Block compression and sharding

XGBoost delivers additional speed gains through **block compression** and **block sharding**.

Block compression helps with computationally expensive disk reading by compressing columns. Block sharding decreases read times by sharding the data into multiple disks that alternate when reading the data.

Accuracy gains

XGBoost adds built-in regularization to achieve accuracy gains beyond gradient boosting. **Regularization** is the process of adding information to reduce variance and prevent overfitting.

Although data may be regularized through hyperparameter fine-tuning, regularized algorithms may also be attempted. For example, `Ridge` and `Lasso` are regularized machine learning alternatives to `LinearRegression`.

XGBoost includes regularization as part of the learning objective, as contrasted with gradient boosting and random forests. The regularized parameters penalize complexity and smooth out the final weights to prevent overfitting. XGBoost is a regularized version of gradient boosting.

In the next section, you will meet the math behind the learning objective of XGBoost, which combines regularization with the loss function. While you don't need to know the math

to use XGBoost effectively, mathematical knowledge may provide a deeper understanding. You can skip the next section if desired.

Analyzing XGBoost parameters

In this section, we will analyze the parameters that XGBoost uses to create state-of-the-art machine learning models with a mathematical derivation.

We will maintain the distinction between parameters and hyperparameters as presented in ***Chapter 2***, *Decision Trees in Depth*. Hyperparameters are chosen before the model is trained, whereas parameters are chosen while the model is being trained. In other words, the parameters are what the model learns from the data.

The derivation that follows is taken from the XGBoost official documentation, *Introduction to Boosted Trees*, at **https://xgboost.readthedocs.io/en/latest/tutorials/model.html**.

Learning objective

The learning objective of a machine learning model determines how well the model fits the data. In the case of XGBoost, the learning objective consists of two parts: the **loss function** and the **regularization term**.

Mathematically, XGBoost's learning objective may be defined as follows:

$$obj(\theta) = l(\theta) + \Omega(\theta)$$

Here, $l(\theta)$ is the loss function, which is the **Mean Squared Error** (**MSE**) for regression, or the log loss for classification, and $\Omega(\theta)$ is the regularization function, a penalty term to prevent over-fitting. Including a regularization term as part of the objective function distinguishes XGBoost from most tree ensembles.

Let's look at the objective function in more detail, by considering the MSE for regression.

Loss function

The loss function, defined as the MSE for regression, can be written in summation notation, as follows:

$$l(\theta) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Here, y_i is the target value for the i_{th} row and $\hat{y}_i$ is the value predicted by the machine learning model for the i_{th} row. The summation symbol, Σ, indicates that all rows are summed starting with $i = 1$ and ending with $i = n$, the number of rows.

The prediction, $\hat{y}_i$, for a given tree requires a function that starts at the tree root and ends at a leaf. Mathematically, this can be expressed as follows:

$$\hat{y}_i = f(x_i), f \in F$$

Here, x_i is a vector whose entries are the columns of the i_{th} row and $f \in F$ means that the function f is a member of F, the set of all possible CART functions. **CART** is an acronym for **Classification And Regression Trees**. CART provides a real value for all leaves, even for classification algorithms.

In gradient boosting, the function that determines the prediction for the i_{th} row includes the sum of all previous functions, as outlined in ***Chapter 4***, *From Gradient Boosting to XGBoost*. Therefore, it's possible to write the following:

$$\hat{y}_i = \sum_{t=1}^{T} f_t(x_i), f_t \in F$$

Here, *T* is the number of boosted trees. In other words, to obtain the prediction for the i_{th} tree, sum the predictions of the previous trees in addition to the prediction for the new tree. The notation $f_t \in F$ insists that the functions belong to F, the set of all possible CART functions.

The learning objective for the t_{th} boosted tree can now be rewritten as follows:

$$obj^{(t)} = \sum_{i=1}^{n} l\left(y_i, \hat{y_i}^t\right) + \sum_{i=1}^{t} \Omega(f_t)$$

Here, $l\left(y_i, \hat{y_i}^t\right)$ is the general loss function of the t_{th} boosted tree and $\Omega(f_t)$ is the regularization term.

Since boosted trees sum the predictions of previous trees, in addition to the prediction of the new tree, it must be the case that $\hat{y_i}^t = \hat{y_i}^{t-1} + f_t(x_i)$. This is the idea behind additive training.

By substituting this into the preceding learning objective, we obtain the following:

$$obj^{(t)} = \sum_{i=1}^{n} l\left(y_i, \hat{y_i}^{t-1} + f_t(x_i)\right) + \Omega(f_t)$$

This can be rewritten as follows for the least square regression case:

$$obj^{(t)} = \sum_{i=1}^{n} \left(y_i - \left(\hat{y_i}^{t-1} + f_t(x_i)\right)\right)^2 + \Omega(f_t)$$

Multiplying the polynomial out, we obtain the following:

$$obj^{(t)} = \sum_{i=1}^{n} 2\left(y_i - \hat{y}_i^{t-1}\right) f_t(x_i) + f_t(x_i)^2 + \Omega(f_t) + C$$

Here, C is a constant term that does not depend on t. In terms of polynomials, this is a quadratic equation with the variable $f_t(x_i)$. Recall that the goal is to find an optimal value of $f_t(x_i)$, the optimal function mapping the roots (samples) to the leaves (predictions).

Any sufficiently smooth function, such as second-degree polynomial (quadratic), can be approximated by a **Taylor polynomial**. XGBoost uses Newton's method with a second-degree Taylor polynomial to obtain the following:

$$obj^{(t)} = \sum_{i=1}^{n} g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 + \Omega(f_t)$$

Here, g_i and h_i can be written as the following partial derivatives:

$$g_i = \partial_{\hat{y}_i^{t-1}} l\left(y_i, \hat{y}_i^{t-1}\right)$$

$$h_i = \partial^2_{\hat{y}_i^{t-1}} l\left(y_i, \hat{y}_i^{t-1}\right)$$

For a general discussion of how XGBoost uses the **Taylor expansion**, check out **https://stats.stackexchange.com/questions/202858/xgboost-loss-function-approximation-with-taylor-expansion**.

XGBoost implements this learning objective function by taking a solver that uses only g_i and h_i as input. Since the loss function is general, the same inputs can be used for regression and classification.

This leaves the regularization function, $\Omega(f_t)$.

Regularization function

Let w be the vector space of leaves. Then, f, the function mapping the tree root to the leaves, can be recast in terms of w, as follows:

$$f_t(x) = w_{q(x)}, w \in R^T, q: R^d \rightarrow \{1,2,\dots,T\}$$

Here, *q* is the function assigning data points to leaves and *T* is the number of leaves.

After practice and experimentation, XGBoost settled on the following as the regularization function where γ and λ are penalty constants to reduce overfitting:

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

Objective function

Combining the loss function with the regularization function, the learning objective function becomes the following:

$$obj^{(t)} = \sum_{i=1}^{n} g_i w_{q(x_i)} + \frac{1}{2} h_i w_{q(x_i)}^2 + \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

We can define the set of indices of data points assigned to the j_{th} leaf as follows:

$$I_j = \{i \mid q(x_i) = j\}$$

The objective function can then be written as follows:

$$obj^{(t)} = \sum_{i \in I_j} g_i w_j + \frac{1}{2} \sum_{i \in I_j} h_i w_j^2 + \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

$$G_j = \sum_{i \in I_j} g_i \qquad H_j = \sum_{i \in I_j} h_i$$

Finally, setting the $G_j = \sum_{i \in I_j} g_i$ and $H_j = \sum_{i \in I_j} h_i$, after rearranging the indices and combining like terms, we obtain the final form of the objective function, which is the following:

$$obj^{(t)} = \sum_{j=1}^{T} [G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2 + \gamma T]$$

Minimizing the objective function by taking the derivative with respect to w_j and setting the left side equal to zero, we obtain the following:

$$w_j = -\frac{G_j}{H_j + \lambda}$$

This can be substituted back into the objection function to give the following:

$$obj^{(t)} = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T$$

This is the result XGBoost uses to determine how well the model fits the data.

Congratulations on making it through a long and challenging derivation!

Building XGBoost models

In the first two sections, you learned how XGBoost works under the hood with parameter derivations, regularization, speed enhancements, and new features such as the **`missing`** parameter to compensate for null values.

In this book, we primarily build XGBoost models with scikit-learn. The scikit-learn XGBoost wrapper was released in 2019. Before full immersion with scikit-learn, building XGBoost models required a steeper learning curve. Converting NumPy arrays to **`dmatrices`**, for instance, was mandatory to take advantage of the XGBoost framework.

In scikit-learn, however, these conversions happen behind the scenes. Building XGBoost models in scikit-learn is very similar to building other machine learning models in scikit-learn, as you have experienced throughout this book. All standard scikit-learn methods, such as `.fit`, and `.predict`, are available, in addition to essential tools such as `train_test_split`, `cross_val_score`, `GridSearchCV`, and `RandomizedSearchCV`.

In this section, you will develop templates for building XGBoost models. Going forward, these templates can be referenced as starting points for building XGBoost classifiers and regressors.

We will build templates for two classic datasets: the **Iris dataset** for classification and the **Diabetes dataset** for regression. Both datasets are small, built into scikit-learn, and have been tested frequently throughout the machine learning community. As part of the model-building process, you will explicitly define default hyperparameters that give XGBoost great scores. These hyperparameters are explicitly defined so that you can learn what they are in preparation for adjusting them going forward.

The Iris dataset

The Iris dataset, a staple of the machine learning community, was introduced by statistician Robert Fischer in 1936. Its easy accessibility, small size, clean data, and symmetry of values have made it a popular choice for testing classification algorithms.

We will introduce the Iris dataset by downloading it directly from scikit-learn using the `datasets` library with the `load_iris()` method, as follows:

```
import pandas as pd
import numpy as np
from sklearn import datasets
iris = datasets.load_iris()
```

Scikit-learn datasets are stored as **NumPy arrays**, the array storage method of choice for machine learning algorithms. `pandas` DataFrames are used more for data analysis and data visualization. Viewing NumPy arrays as DataFrames requires the `pandas DataFrame` method. This scikit-learn dataset is split into predictor and target columns in advance. Bringing them together requires concatenating the NumPy arrays with the code `np.c_` before conversion. Column names are also added, as follows:

```
df = pd.DataFrame(data= np.c_[iris['data'], iris['target']],columns=
iris['feature_names'] + ['target'])
```

You can view the first five rows of the DataFrame using `df.head()`:

```
df.head()
```

The resulting DataFrame will look like this:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0.0
1	4.9	3.0	1.4	0.2	0.0
2	4.7	3.2	1.3	0.2	0.0
3	4.6	3.1	1.5	0.2	0.0
4	5.0	3.6	1.4	0.2	0.0

Figure 5.1 – The Iris dataset

The predictor columns are self-explanatory, measuring sepal and petal length and width. The target column, according to the scikit-learn documentation, **https://scikit-learn.org/stable/auto_examples/datasets/plot_iris_dataset.html**, consists of three different iris flowers, **setosa**, **versicolor**, and **virginica**. There are 150 rows.

To prepare the data for machine learning, import **`train_test_split`**, then split the data accordingly. You can use the original NumPy arrays, **`iris['data']`** and **`iris['target']`**, as inputs for **`train_test_split`**:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(iris['data'], iris['target'], random_state=2)
```

Now that we have split the data, let's build the classification template.

XGBoost classification template

The following template is for building an XGBoost classifier, assuming the dataset has already been split into **`X_train`**, **`X_test`**, **`y_train`**, and **`y_test`** sets:

1. Import **`XGBClassifier`** from the **`xgboost`** library:
   ```
   from xgboost import XGBClassifier
   ```
2. Import a classification scoring method as needed.
   While **`accuracy_score`** is standard, other scoring methods, such as **`auc`** (**Area Under Curve**), will be discussed later:
   ```
   from sklearn.metrics import accuracy_score
   ```
3. Initialize the XGBoost classifier with hyperparameters.
   Fine-tuning hyperparameters is the focus of ***Chapter 6***, *XGBoost Hyperparameters*. In this chapter, the most important default hyperparameters are explicitly stated ahead:
   ```
   xgb = XGBClassifier(booster='gbtree', objective='multi:softprob', max_depth=6, learning_rate=0.1, n_estimators=100, random_state=2, n_jobs=-1)
   ```
   The brief descriptions of the preceding hyperparameters are as follows:

a) **`booster='gbtree'`**: The booster is the **base learner**. It's the machine learning model that is constructed during every round of boosting. You may have guessed that **`'gbtree'`** stands for gradient boosted tree, the XGBoost default base learner. It's uncommon but possible to work with other base learners, a strategy we employ in ***Chapter 8***, *XGBoost Alternative Base Learners*.
b) **`objective='multi:softprob'`**: Standard options for the objective can be viewed in the XGBoost official documentation, **https://xgboost.readthedocs.io/en/latest/parameter.html**, under *Learning Task Parameters*. The **`multi:softprob`** objective is a standard alternative to **`binary:logistic`** when the dataset includes multiple classes. It computes the probabilities of classification and chooses the highest one. If not explicitly stated, XGBoost will often find the right objective for you.
c) **`max_depth=6`**: The **`max_depth`** of a tree determines the number of branches each tree has. It's one of the most important hyperparameters in making balanced predictions. XGBoost uses a default of **`6`**, unlike random forests, which don't provide a value unless explicitly programmed.
d) **`learning_rate=0.1`**: Within XGBoost, this hyperparameter is often referred to as **`eta`**. This hyperparameter limits the variance by reducing the weight of each tree to the given percentage. The **`learning_rate`** hyperparameter was explored in detail in ***Chapter 4***, *From Gradient Boosting to XGBoost*.
e) **`n_estimators=100`**: Popular among ensemble methods, **`n_estimators`** is the number of boosted trees in the model. Increasing this number while decreasing **`learning_rate`** can lead to more robust results.

4. Fit the classifier to the data.
   This is where the magic happens. The entire XGBoost system, the details explored in the previous two sections, the selection of optimal parameters, including regularization constraints, and speed enhancements, such as the approximate split-finding algorithm, and blocking and sharding all occur during this one powerful line of scikit-learn code:
   ```
   xgb.fit(X_train, y_train)
   ```
5. Predict the *y* values as **`y_pred`**:
   ```
   y_pred = xgb.predict(X_test)
   ```
6. Score the model by comparing **`y_pred`** against **`y_test`**:
   ```
   score = accuracy_score(y_pred, y_test)
   ```
7. Display your results:
   ```
   print('Score: ' + str(score))
   Score: 0.9736842105263158
   ```

Unfortunately, there is no official list of Iris dataset scores. There are too many to compile in one place. An initial score of **`97.4`** percent on the Iris dataset using default hyperparameters is very good (see **https://www.kaggle.com/c/serpro-iris/leaderboard**).

The XGBoost classifier template provided in the preceding paragraphs is not meant to be definitive, but rather a starting point going forward.

The Diabetes dataset

Now that you are becoming familiar with scikit-learn and XGBoost, you are developing the ability to build and score XGBoost models fairly quickly. In this section, an XGBoost regressor template is provided using **`cross_val_score`** with scikit-learn's Diabetes dataset.

Before building the template, import the predictor columns as **`X`** and the target columns as **`y`**, as follows:

```
X,y = datasets.load_diabetes(return_X_y=True)
```

Now that we have imported the predictor and target columns, let's start building the template.

The XGBoost regressor template (cross-validation)

Here are the essential steps to build an XGBoost regression model in scikit-learn using cross-validation, assuming that the predictor columns, **`X`**, and the target column, **`y`**, have been defined:

1. Import **`XGBRegressor`** and **`cross_val_score`**:
   ```
   from sklearn.model_selection import cross_val_score
   from xgboost import XGBRegressor
   ```
2. Initialize **`XGBRegressor`**.
   Here, we initialize **`XGBRegressor`** with **`objective='reg:squarederror'`**, the MSE. The most important hyperparameter defaults are explicitly given:
   ```
   xgb = XGBRegressor(booster='gbtree', objective='reg:squarederror',
   max_depth=6, learning_rate=0.1, n_estimators=100, random_state=2, n_jobs=-
   1)
   ```
3. Fit and score the regressor with **`cross_val_score`**.
   With **`cross_val_score`**, fitting and scoring are done in one step using the model, the predictor columns, the target column, and the scoring as inputs:
   ```
   scores = cross_val_score(xgb, X, y, scoring='neg_mean_squared_error', cv=5)
   ```
4. Display the results.
   Scores for regression are commonly displayed as the **Root Mean Squared Error** (**RMSE**) to keep the units the same:
   ```
   rmse = np.sqrt(-scores)
   print('RMSE:', np.round(rmse, 3))
   print('RMSE mean: %0.3f' % (rmse.mean()))
   ```
   The result is as follows:
   ```
   RMSE: [63.033 59.689 64.538 63.699 64.661]
   RMSE mean: 63.124
   ```

Without a baseline of comparison, we have no idea what that score means. Converting the target column, **`y`**, into a **`pandas`** DataFrame with the **`.describe()`** method will give the quartiles and the general statistics of the predictor column, as follows:

```
pd.DataFrame(y).describe()
```

Here is the expected output:

	0
count	442.000000
mean	152.133484
std	77.093005
min	25.000000
25%	87.000000
50%	140.500000
75%	211.500000
max	346.000000

Figure 5.2 – Describing the statistics of y, the Diabetes target column

A score of `63.124` is less than 1 standard deviation, a respectable result.

You now have XGBoost classifier and regressor templates that can be used for building models going forward.

Now that you are accustomed to building XGBoost models in scikit-learn, it's time for a deep dive into high energy physics.

Finding the Higgs boson – case study

In this section, we will review the Higgs Boson Kaggle Competition, which brought XGBoost into the machine learning spotlight. In order to set the stage, the historical background is given before moving on to model development. The models that we build include a default model provided by XGBoost at the time of the competition and a reference to the winning solution provided by Gabor Melis. Kaggle accounts are not required for this text, so we will not take the time to show you how to make submissions. We have provided guidelines if you are interested.

Physics background

In popular culture, the Higgs boson is known as the *God particle*. Theorized by Peter Higgs in 1964, the Higgs boson was introduced to explain why particles have mass.

The search to find the Higgs boson culminated in its discovery in 2012 in the **Large Hadron Collider** at CERN (Geneva, Switzerland). Nobel Prizes were awarded and the Standard Model of

physics, the model that accounts for every force known to physics except for gravity, stood taller than ever before.

The Higgs boson was discovered by smashing protons into each other at extremely high speeds and observing the results. Observations came from the **ATLAS** detector, which records data resulting from *hundreds of millions of proton-proton collisions per second*, according to the competition's technical documentation, *Learning to discover: the Higgs boson machine learning challenge*, **https://higgsml.lal.in2p3.fr/files/2014/04/documentation_v1.8.pdf**.

After discovering the Higgs boson, the next step was to precisely measure the characteristics of its decay. The ATLAS experiment found the Higgs boson decaying into two **tau** particles from data wrapped in background noise. To better understand the data, ATLAS called upon the machine learning community.

Kaggle competitions

The Kaggle competition is a machine learning competition designed to solve a particular problem. Machine learning competitions became famous in 2006 when Netflix offered 1 million dollars to anyone who could improve upon their movie recommendations by 10%. In 2009, the 1 million dollar prize was awarded to *BellKor*'s *Pragmatic Chaos* team (**https://www.wired.com/2009/09/bellkors-pragmatic-chaos-wins-1-million-netflix-prize/**).

Many businesses, computer scientists, mathematicians, and students became aware of the increasing value that machine learning held in society. Machine learning competitions became hot, with mutual benefits going to company hosts and machine learning practitioners. Starting in 2010, many early adopters went to Kaggle to try their hand at machine learning competitions.

In 2014, Kaggle announced the *Higgs Boson Machine Learning Challenge* with ATLAS (**https://www.kaggle.com/c/higgs-boson**). With a $13,000 prize pool, 1,875 teams entered the competition.

In Kaggle competitions, training data is provided, along with a required scoring method. Teams build machine learning models on the training data before submitting their results. The target column of the test data is not provided. Multiple submissions are permitted, however, and scores are returned so that competitors can improve upon their models before the final date.

Kaggle competitions are fertile ground for testing machine learning algorithms. Unlike in industry, Kaggle competitions draw thousands of competitors, making the machine learning models that win prizes very well tested.

XGBoost and the Higgs challenge

XGBoost was released to the general public on March 27, 2014, 6 months before the Higgs challenge. In the competition, XGBoost soared, helping competitors climb the Kaggle leaderboard while saving valuable time.

Let's access the data to see what the competitors were working with.

Data

Instead of using the data provided by Kaggle, we use the original data provided by the CERN open data portal where it originated: **http://opendata.cern.ch/record/328**. The difference between the CERN data and the Kaggle data is that the CERN dataset is significantly larger. We will select the first 250,000 rows and make some modifications to match the Kaggle data.

You can download the CERN Higgs boson dataset directly from **https://github.com/PacktPublishing/Hands-On-Gradient-Boosting-with-XGBoost-and-Scikit-learn/tree/master/Chapter05**.

Read the **`atlas-higgs-challenge-2014-v2.csv.gz`** file into a **`pandas`** DataFrame. Please note that we are selecting the first 250,000 rows only, and the **`compression=gzip`** parameter is used since the dataset is zipped as a **`csv.gz`** file. After accessing the data, view the first five rows, as follows:

```
df = pd.read_csv('atlas-higgs-challenge-2014-v2.csv.gz', nrows=250000,
compression='gzip')
df.head()
```

The far-right columns of the output should be as shown in the following screenshot:

PRI_jet_leading_phi	PRI_jet_subleading_pt	PRI_jet_subleading_eta	PRI_jet_subleading_phi	PRI_jet_all_pt	Weight	Label	KaggleSet	KaggleWeight
0.444	46.062	1.24	-2.475	113.497	0.000814	s	t	0.002653
1.158	-999.000	-999.00	-999.000	46.226	0.681042	b	t	2.233584
-2.028	-999.000	-999.00	-999.000	44.251	0.715742	b	t	2.347389
-999.000	-999.000	-999.00	-999.000	-0.000	1.660654	b	t	5.446378
-999.000	-999.000	-999.00	-999.000	0.000	1.904263	b	t	6.245333

Figure 5.3 – CERN Higgs boson data – Kaggle columns included

Notice the **`Kaggleset`** and **`KaggleWeight`** columns. Since the Kaggle dataset was smaller, Kaggle used a different number for their weight column which is denoted in the preceding diagram as **`KaggleWeight`**. The **`t`** value under **`Kaggleset`** indicates that it's part of the training set for the Kaggle dataset. In other words, these two columns, **`Kaggleset`** and **`KaggleWeight`**, are columns in the CERN dataset designed to include information that will be used for the

Kaggle dataset. In this chapter, we will restrict our subset of the CERN data to the Kaggle training set.

To match the Kaggle training data, let's delete the **`Kaggleset`** and **`Weight`** columns, convert **`KaggleWeight`** into **`'Weight'`**, and move the **`'Label'`** column to the last column, as follows:

```
del df[‹Weight›]
del df[‹KaggleSet›]
df = df.rename(columns={«KaggleWeight»: «Weight»})
```

One way to move the **`Label`** column is to store it as a variable, delete the column, and add a new column by assigning it to the new variable. Whenever assigning a new column to a DataFrame, the new column appears at the end:

```
label_col = df['Label']
del df['Label']
df['Label'] = label_col
```

Now that all changes have been made, the CERN data matches the Kaggle data. Go ahead and view the first five rows:

```
df.head()
```

Here is the left side of the expected output:

	EventId	DER_mass_MMC	DER_mass_transverse_met_lep	DER_mass_vis	DER_pt_h	DER_deltaeta_jet_jet	DER_mass_jet_jet	DER_prodeta_jet_jet	DER_deltar
0	100000	138.470	51.655	97.827	27.980	0.91	124.711	2.666	
1	100001	160.937	68.768	103.235	48.146	-999.00	-999.000	-999.000	
2	100002	-999.000	162.172	125.953	35.635	-999.00	-999.000	-999.000	
3	100003	143.905	81.417	80.943	0.414	-999.00	-999.000	-999.000	
4	100004	175.864	16.915	134.805	16.405	-999.00	-999.000	-999.000	

5 rows × 33 columns

Figure 5.4 – CERN Higgs boson data – physics columns

Many columns are not shown, and an unusual value of **`-999.00`** occurs in multiple places.

The columns beyond **`EventId`** include variables prefixed with **`PRI`**, which stands for *primitives*, which are values directly measured by the detector during collisions. By contrast, columns labeled **`DER`** are numerical derivations from these measurements.

All column names and types are revealed by **`df.info()`**:

```
df.info()
```

Here is a sample of the output, with the middle columns truncated to save space:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 250000 entries, 0 to 249999
Data columns (total 33 columns):
#   Column                      Non-Null Count   Dtype
---  ------                      --------------   -----
0   EventId                     250000 non-null  int64
1   DER_mass_MMC                250000 non-null  float64
2   DER_mass_transverse_met_lep 250000 non-null  float64
3   DER_mass_vis                250000 non-null  float64
4   DER_pt_h                    250000 non-null  float64
…
28  PRI_jet_subleading_eta      250000 non-null  float64
29  PRI_jet_subleading_phi      250000 non-null  float64
30  PRI_jet_all_pt              250000 non-null  float64
31  Weight                      250000 non-null  float64
32  Label                       250000 non-null  object
dtypes: float64(30), int64(3)
memory usage: 62.9 MB
```

All columns have non-null values, and only the final column, **`Label`**, is non-numerical. The columns can be grouped as follows:

- Column `0` : **`EventId`** – irrelevant for the machine learning model.
- Columns **`1-30`**: Physics columns derived from LHC collisions. Details for these columns can be found in the link to the technical documentation at **http://higgsml.lal.in2p3.fr/documentation**. These are the machine learning predictor columns.
- Column `31` : **`Weight`** – this column is used to scale the data. The issue here is that Higgs boson events are very rare, so a machine learning model with 99.9 percent accuracy may not be able to find them. Weights compensate for this imbalance, but weights are not available for the test data. Strategies for dealing with weights will be discussed later in this chapter, and in ***Chapter 7***, *Discovering Exoplanets with XGBoost*.
- Column `32`: **`Label`** – this is the target column, labeled **`s`** for signal and **`b`** for background. The training data has been simulated from real data, so there are many more signals than otherwise would be found. The signal is the occurrence of the Higgs boson decay.

The only issue with the data is that the target column, **`Label`**, is not numerical. Convert the **`Label`** column into a numerical column by replacing the **`s`** values with `1` and the **`b`** values with `0`, as follows:

```
df['Label'].replace(('s', 'b'), (1, 0), inplace=True)
```

Now that all columns are numerical with non-null values, you can split the data into predictor and target columns. Recall that the predictor columns are indexed 1–30 and the target column is the last column, indexed `32` (or -1). Note that the **`Weight`** column should not be included because it's not available for the test data:

```
X = df.iloc[:,1:31]
y = df.iloc[:,-1]
```

Scoring

The Higgs Challenge is not your average Kaggle competition. In addition to the difficulty of understanding high energy physics for feature engineering (a route we will not pursue), the scoring method is not standard. The Higgs Challenge requires optimizing the **Approximate Median Significance** (**AMS**).

The AMS is defined as follows:

$$\sqrt{2((s+b+b_{reg})ln(1+\frac{s}{b+b_{reg}}-s)}$$

Here, s is the true positive rate, b is the false positive rate, and b_{reg} is a constant regularization term given as `10`.

Fortunately, XGBoost provided an AMS scoring method for the competition, so it does not need to be formally defined. A high AMS results from many true positives and few false negatives. Justification for the AMS instead of other scoring methods is given in the technical documentation at **http://higgsml.lal.in2p3.fr/documentation**.

Tip

It's possible to build your own scoring methods, but it's not usually needed. In the rare event that you need to build your own scoring method, you can check out **https://scikit-learn.org/stable/modules/model_evaluation.html** *for more information.*

Weights

Before building a machine learning model for the Higgs boson, it's important to understand and utilize weights.

In machine learning, weights can be used to improve the accuracy of imbalanced datasets. Consider the `s` (signal) and `b` (background) columns in the Higgs challenge. In reality, `s` << `b`, so signals are very rare among the background noise. Let's say, for example, that signals are 1,000 times rarer than background noise. You can create a weight column where `b` = 1 and `s` = 1/1000 to compensate for this imbalance.

According to the technical documentation of the competition, the weight column is a **scale factor** that, when summed, gives the expected number of signal and background events during the time of data collection in 2012. This means that weights are required for the predictions to represent reality. Otherwise, the model will predict way too many **`s`** (signal) events.

The weights should first be scaled to match the test data since the test data provides the expected number of signal and background events generated by the test set. The test data has 550,000 rows, more than twice the 250,000 rows (**`len(y)`**) provided by the training data. Scaling weights to match the test data can be achieved by multiplying the weight column by the percentage of increase, as follows:

```
df['test_Weight'] = df['Weight'] * 550000 / len(y)
```

Next, XGBoost provides a hyperparameter, **`scale_pos_weight`**, which takes the scaling factor into account. The scaling factor is the sum of the weights of the background noises divided by the sum of the weight of the signal. The scaling factor can be computed using **`pandas`** conditional notation, as follows:

```
s = np.sum(df[df['Label']==1]['test_Weight'])
b = np.sum(df[df['Label']==0]['test_Weight'])
```

In the preceding code, **`df[df['Label']==1]`** narrows the DataFrame down to rows where the **`Label`** column equals **`1`**, then **`np.sum`** adds the values of these rows using the **`test_Weight`** column.

Finally, to see the actual rate, divide **`b`** by **`s`**:

```
b/s
593.9401931492318
```

In summary, the weights represent the expected number of signal and background events generated by the data. We scale the weights to match the size of the test data, then divide the sum of the background weights by the sum of the signal weights to establish the **`scale_pos_weight=b/s`** hyperparameter.

Tip

For a more detailed discussion on weights, check out the excellent introduction from KDnuggets at **https://www.kdnuggets.com/2019/11/machine-learning-what-why-how-weighting.html**.

The model

It's time to build an XGBoost model to predict the signal – that is, the simulated occurrences of the Higgs boson decay.

At the time of the competition, XGBoost was new, and the scikit-learn wrapper was not yet available. Even today (2020), the majority of information online about implementing XGBoost in Python is pre-scikit-learn. Since you are likely to encounter the pre-scikit-learn XGBoost Python API online, and this is what all competitors used in the Higgs Challenge, we present code using the original Python API in this chapter only.

Here are the steps to build an XGBoost model for the Higgs Challenge:

1. Import **`xgboost`** as **`xgb`**:
   ```
   import xgboost as xgb
   ```
2. Initialize the XGBoost model as a **DMatrix** with the missing values and weights filled in.
   All XGBoost models were initialized as a DMatrix before scikit-learn. The scikit-learn wrapper automatically converts the data into a DMatrix for you. The sparse matrices that XGBoost optimizes for speed are DMatrices.
   According to the documentation, all values set to **`-999.0`** are unknown values. Instead of converting these values into the median, mean, mode, or other null replacement, in XGBoost, unknown values can be set to the **`missing`** hyperparameter. During the model build phase, XGBoost automatically chooses the value leading to the best split.
3. The **`weight`** hyperparameter can equal the new column, **`df['test_Weight']`**, as defined in the **`weight`** section:
   ```
   xgb_clf = xgb.DMatrix(X, y, missing=-999.0, weight=df['test_Weight'])
   ```
4. Set additional hyperparameters.
   The hyperparameters that follow are defaults provided by XGBoost for the competition:
   a) Initialize a blank dictionary called **`param`**:
   ```
   param = {}
   ```
   b) Define the objective as **`'binary:logitraw'`**.
   This means a binary model is created from logistic regression probabilities. This objective defines the model as a classifier and allows a ranking of the target column, which is required of submissions for this particular Kaggle competition:
   ```
   param['objective'] = 'binary:logitraw'
   ```
   c) Scale the positive examples using the background weights divided by the signal weights. This will help the model perform better on the test set:
   ```
   param['scale_pos_weight'] = b/s
   ```
   d) The learning rate, **`eta`**, is given as **`0.1`**:
   ```
   param['eta'] = 0.1
   ```
   e) **`max_depth`** is given as **`6`**:
   ```
   param['max_depth'] = 6
   ```
   f) Set the scoring method as **`'auc'`** for display purposes:
   ```
   param['eval_metric'] = 'auc'
   ```
   Although the AMS score will be printed, the evaluation metric is given as **`auc`**, which stands for **Area Under Curve**. **`auc`** is the true positive versus false positive curve that is perfect when it equals **`1`**. Similar to accuracy, **`auc`** is a standard scoring metric for

classification, although it's often superior to accuracy since accuracy is limited for imbalanced datasets, as discussed in ***Chapter 7***, *Discovering Exoplanets with XGBoost*.

5. Create a list of parameters that includes the preceding items, along with the evaluation metric (`auc`) and `ams@0.15`, XGBoost's implementation of the AMS score using a 15% threshold:
```
plst = list(param.items())+[('eval_metric', 'ams@0.15')]
```
6. Create a watchlist that includes the initialized classifier and `'train'` so that you can view scores as the trees continue to boost:
```
watchlist = [ (xg_clf, 'train') ]
```
7. Set the number of boosting rounds to `120`:
```
num_round = 120
```
8. Train and save the model. Train the model by placing the parameter list, the classifier, the number of rounds, and the watchlist as inputs. Save the model using the `save_model` method so that you do not have to go through a time-consuming training process a second time. Then, run the code and watch how the scores improve as the trees are boosted:
```
print ('loading data end, start to boost trees')
bst = xgb.train( plst, xgmat, num_round, watchlist )
bst.save_model('higgs.model')
print ('finish training')
```

The end of your results should have the following output:

```
[110] train-auc:0.94505 train-ams@0.15:5.84830
[111] train-auc:0.94507 train-ams@0.15:5.85186
[112] train-auc:0.94519 train-ams@0.15:5.84451
[113] train-auc:0.94523 train-ams@0.15:5.84007
[114] train-auc:0.94532 train-ams@0.15:5.85800
[115] train-auc:0.94536 train-ams@0.15:5.86228
[116] train-auc:0.94550 train-ams@0.15:5.91160
[117] train-auc:0.94554 train-ams@0.15:5.91842
[118] train-auc:0.94565 train-ams@0.15:5.93729
[119] train-auc:0.94580 train-ams@0.15:5.93562
finish training
```

Congratulations on building an XGBoost classifier that can predict Higgs boson decay!

The model performs with `94.58` percent `auc`, and an AMS of `5.9`. As far as the AMS is concerned, the top values of the competition were in the upper threes. This model achieves an AMS of around `3.6` when submitted with the test data.

The model that you just built was provided as a baseline by Tanqi Chen for XGBoost users during the competition. The winner of the competition, Gabor Melis, used this baseline to build his model. As can be seen from viewing the winning solution at **https://github.com/melisgl/higgsml** and clicking on **xgboost-scripts**, changes made to the baseline model are not significant. Melis, like most Kaggle competitors, also performed feature engineering to add more relevant columns to the data, a practice we will address in ***Chapter 9***, *XGBoost Kaggle Masters*.

It is possible to build and train your own model after the deadline and submit it through Kaggle. For Kaggle competitions, submissions must be ranked, properly indexed, and delivered with the Kaggle API topics that require further explanation. If you want to submit models for the actual competition, the XGBoost ranking code, which you may find helpful, is available at **https://github.com/dmlc/xgboost/blob/master/demo/kaggle-higgs/higgs-pred.py**.

Summary

In this chapter, you learned how XGBoost was designed to improve the accuracy and speed of gradient boosting with missing values, sparse matrices, parallel computing, sharding, and blocking. You learned the mathematical derivation behind the XGBoost objective function that determines the parameters for gradient descent and regularization. You built `XGBClassifier` and `XGBRegressor` templates from classic scikit-learn datasets, obtaining very good scores. Finally, you built the baseline model provided by XGBoost for the Higgs Challenge that led to the winning solution and lifted XGBoost into the spotlight.

Now that you have a solid understanding of the overall narrative, design, parameter selection, and model-building templates of XGBoost, in the next chapter, you will fine-tune XGBoost's hyperparameters to achieve optimal scores.